Alexander Baron

CSCI-611 A3: Small Object Detection Using YOLO

Task 1: Set Up YOLO Environment

The first task of this Assignment involved configuring the YOLOv8 environment. By leveraging the *ultralytics* library, the framework was initialized and the pre-trained weights were loaded using the *YOLO()* constructor. For this Assignment YOLOv8n (Nano) variant was used. Figure 1.1 shows that the model architecture consists of approximately 3.2 million parameters. This lightweight structure allows for high-speed inference, though it presents unique challenges for the detection of small, distant objects like traffic signals, which often occupy only a few pixels of the input frame.

```
                 from  n    params  module                                       arguments
  0                 -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                 -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                 -1  1      7360  ultralytics.nn.modules.block.C2f             [32, 32, 1, True]
  3                 -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
  4                 -1  2     49664  ultralytics.nn.modules.block.C2f             [64, 64, 2, True]
  5                 -1  1     73984  ultralytics.nn.modules.conv.Conv             [64, 128, 3, 2]
  6                 -1  2    197632  ultralytics.nn.modules.block.C2f             [128, 128, 2, True]
  7                 -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                 -1  1    460288  ultralytics.nn.modules.block.C2f             [256, 256, 1, True]
  9                 -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                 -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 11            [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 12                 -1  1    148224  ultralytics.nn.modules.block.C2f             [384, 128, 1]
 13                 -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 14            [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 15                 -1  1     37248  ultralytics.nn.modules.block.C2f             [192, 64, 1]
 16                 -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 17           [-1, 12]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 18                 -1  1    123648  ultralytics.nn.modules.block.C2f             [192, 128, 1]
 19                 -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 20            [-1, 9]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 21                 -1  1    493056  ultralytics.nn.modules.block.C2f             [384, 256, 1]
 22        [15, 18, 21]  1    755797  ultralytics.nn.modules.head.Detect           [23, 16, None, [64, 128, 256]]
Model summary: 130 layers, 3,015,333 parameters, 3,015,317 gradients, 8.2 GFLOPs
```

Figure 1.1 Model Architecture After Being Loaded

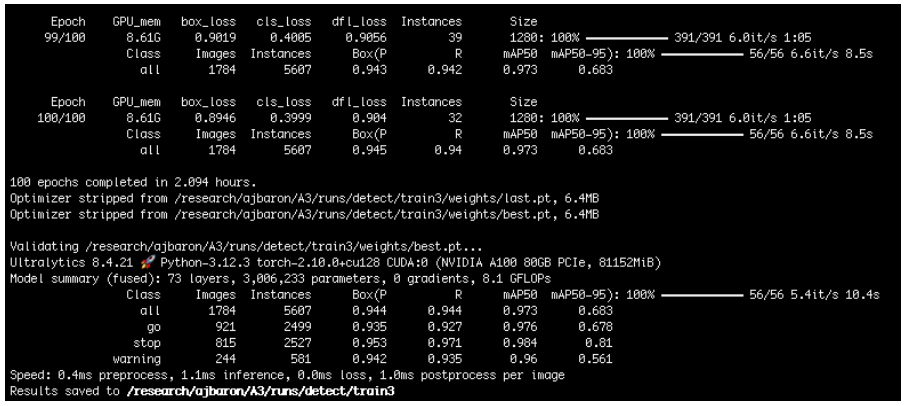
Task 2: Prepare the Dataset

To ensure the model can generalize across varying lighting conditions and distances, the LISA (Labratory for Intelligent and Safe Automobiles) Traffic Sign Dataset was selected. This dataset is particularly well-suited for this task as it contains over 6,000 frames of US traffic signs under diverse real-world conditions.

The dataset was managed and exported via Roboflow**,** which facilitated the conversion of raw annotations into the YOLOv8 PyTorch format. This format consists of a collection of .txt files containing normalized bounding box coordinated *[class_id,* x_{center} *,* y_{center}*, width, height],* and corresponding data.uaml configuration file. This file is essential for the Ultralytics trainer to map the training/validation paths and define the specific class indices for the traffic signals.

Task 3: Train the YOLO Environment for Small Object Detection

Initial training was attempted on a local CPU environment (Apple M-series), but processing speed were insufficient for deep learning at scale. The training was subsequently migrated to a high-performance GPU server.Figure 3.1 shows the training results, showing the significant performance gains, completing 100 epochs in 2 hours, a performance increase that made testing more feasible.



```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
     99/100      8.61G     0.9019     0.4005     0.9056         39       1280: 100% ━━━━━━━━━━━━ 391/391 6.0it/s 1:05
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 56/56 6.6it/s 8.5s
                   all       1784       5607      0.943      0.942      0.973      0.683

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
    100/100      8.61G     0.8946     0.3999      0.904         32       1280: 100% ━━━━━━━━━━━━ 391/391 6.0it/s 1:05
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 56/56 6.6it/s 8.5s
                   all       1784       5607      0.945       0.94      0.973      0.683

100 epochs completed in 2.094 hours.
Optimizer stripped from /research/ajbaron/A3/runs/detect/train3/weights/last.pt, 6.4MB
Optimizer stripped from /research/ajbaron/A3/runs/detect/train3/weights/best.pt, 6.4MB

Validating /research/ajbaron/A3/runs/detect/train3/weights/best.pt...
Ultralytics 8.4.21 🚀 Python-3.12.3 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100 80GB PCIe, 81152MiB)
Model summary (fused): 73 layers, 3,006,233 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 56/56 5.4it/s 10.4s
                   all       1784       5607      0.944      0.944      0.973      0.683
                    go        921       2499      0.935      0.927      0.976      0.678
                  stop        815       2527      0.953      0.971      0.984       0.81
               warning        244        581      0.942      0.935       0.96      0.561
Speed: 0.4ms preprocess, 1.1ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to /research/ajbaron/A3/runs/detect/train3
```

Figure 1.2 Training Results on GPU

Figure 3.2 shows steady convergence in our loss curves., both the box loss, and classification loss for the validation set decreased consistently, indicating the model was effectively learning to localize and categorize traffic signals without significant overfitting.

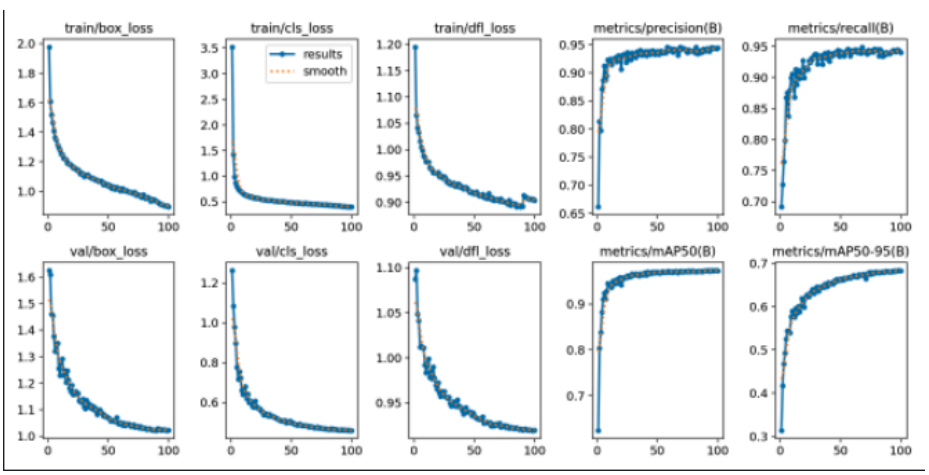


Figure 1.2: Training and Validation Loss Curves showing convergence

Upon completion of training, the model's performance was evaluated using standard computer vision metrics. The model achieved a balanced F1 score of 0.94 at a confidence threshold of 0.372, which is show in Figure 3.3. Mean Average Precision (mAP) at 0.5 IoU reached 0.973, demonstrating exceptionally high reliability in detection.

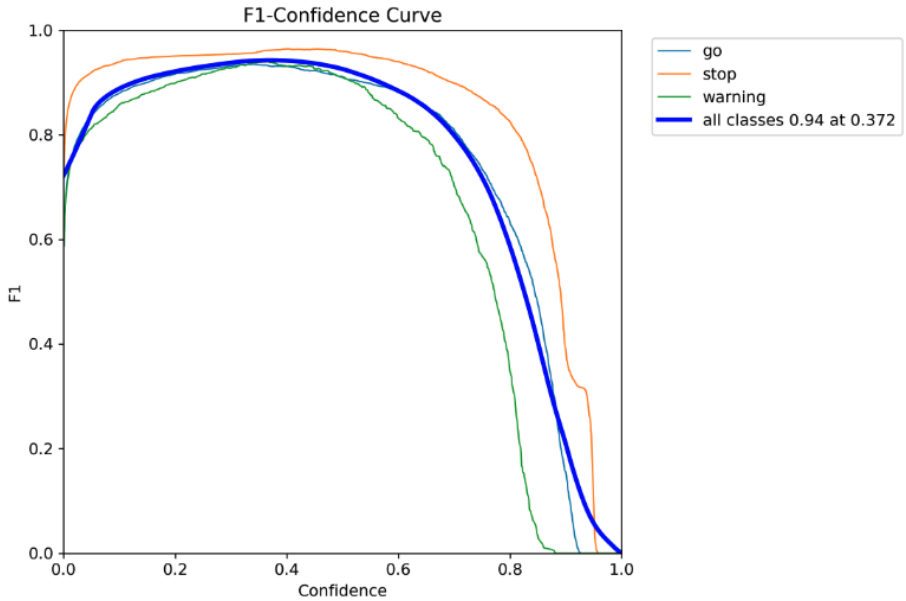


Figure 3.3: F1 Confidence curve showing peak performance at 0.372 confidence

The confusion matrix in Figure 3.4 provides insigh into the models decision making. We note that the model was able to achieve a high accuracy on Go (2419 Correct), and Stop (2473 Correct). There were 259 instances where the Go signal was predicted but the ground truth was background. Suggesting that in complex, cluttered environments, some environmental features are occasioanly misinterpreted as active .signals. While Warning signals had a smaller sample size, the model still identified 556 instances, while maintaining a precision score of 0.942.

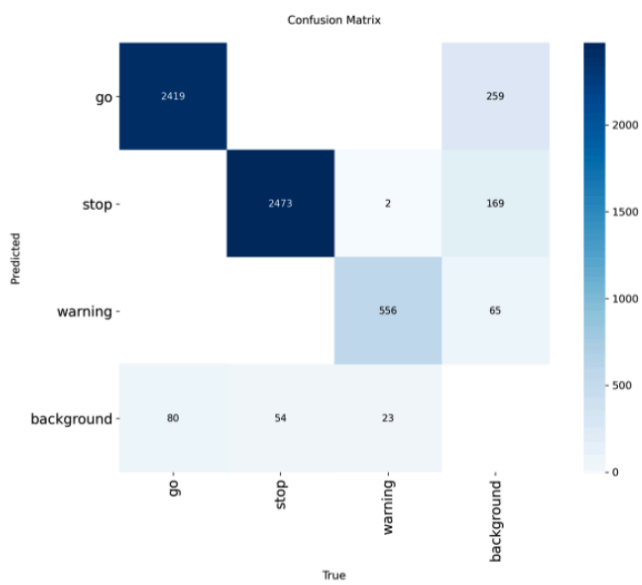


Figure 3.4: Confusion Matrix of accuracy across traffic signals

Task 4: Test on Video or Image Sequence

The model was deployed on video sequences captured from vehicle-mounted cameras on local streets. Serving to validate the model's performance in high-glare real world lighting conditions which differ significantly from the curated LISA dataset. We see the model identified a green singal as Go with the confidence of 0.56 in Figure 4.1. The frame in Figure 4.2 shows the model correctly identifies a yellow signal as warning with confidence of 0.59.

Figure 4.1: Green Singal being Identified as Go

Figure 4.2: Yellow Singal being Identified as Warning

While the model is successfully identifying the signal, several observations point out, The detections appear in the 0.5 - 0.6 range. While this is above our 0.5 threshold, it suggest environment noise such as

the bright sky or reflective surfaces on commercial buildings are competing with the small pixel clusters of the traffic lights.

To ensure small objects weren't suppressed by larger bounding boxed, the NMS Intersection over Union (IoU) threshold was kept conservative. Ensuring the model didnt over filter valid detections that are close to other verical structures. Compared to the Base pre-trained on LISA dataset, this fine-tune model shows significantly high recall for traffic specific states. The base model often classifieds theses as "traffic light", where as our version provides the necessary semantic states (Go, Stop, Warning).